

Generative AI - Fresher's Complete Guide (Hinglish)

Same complete topic coverage, but explanations, examples aur interview answers easy Hinglish mein.

Target: Software/AI fresher | **Focus:** concepts + implementation + interview readiness

How to Use This Guide

Fresher ko GenAI seekhne ke liye ML researcher banna zaroori nahi hai. Practical target yeh hai ki aap LLM stack ko samjho, model ko API se call kar pao, achhe prompts likho, embeddings aur vector search samjho, RAG build karo, tool calling aur agents ka basic samjho, aur apne project ko reliable aur secure bana sako.

- **Core programming:** Python ya JavaScript, HTTP/REST APIs, JSON, Git, environment variables.
- **AI core:** tokens, embeddings, transformers, attention, LLMs, inference, context windows.
- **Application layer:** prompting, structured output, function/tool calling, RAG, evaluation, safety.
- **Engineering:** caching, rate limiting, logging, retries, cost/latency, security.
- **Interview layer:** resume par jo bhi keyword likha hai, usko implementation ke saath explain kar pao.

1. What Is Generative AI?

Generative AI aise AI systems ko bolte hain jo naya content generate kar sakte hain - jaise text, code, images, audio, video, ya structured data. LLMs generative models ka ek major class hain jo language aur code ke liye specially use hote hain.

AI -> ML -> Deep Learning -> Generative AI

Term	Simple Hinglish meaning	Example
AI	Broad field jahan machines intelligent tasks perform karti hain	Recommendation system
ML	Data se patterns learn karne wale systems	Spam classifier
Deep Learning	Multiple neural-network layers use karne wala ML	Image classifier
Generative AI	Naya content generate karne wale models	LLM se answer likhna
LLM	Bahut large language model jo text/code generate karta hai	GPT-family model

2. The LLM Mental Model

High level par flow simple hai: user ka text -> tokens -> model ke transformer layers -> next token ki probability -> decoding se token select -> process repeat hota hai, jab tak response complete na ho.

Important terms

- **Token:** Model text ko chhote units mein todta hai. Token full word, subword ya punctuation bhi ho sakta hai.
- **Context window:** Ek request mein model kitna context process kar sakta hai.
- **Inference:** Trained model ko run karke prediction/answer generate karna.
- **Temperature:** Output mein randomness control karta hai. Low temperature generally zyada deterministic hoti hai.

- **Top-p:** Sampling ko high-probability tokens ke selected probability mass tak limit karta hai.
- **Hallucination:** Model ka unsupported, incorrect, ya fabricated answer.

Interview-ready: "LLM context ko dekhkar next token predict karta hai. Is process ko repeatedly perform karke model text ya code generate karta hai."

3. Transformers and Attention

Why transformers matter

Modern LLMs largely Transformer architecture par based hain. Transformer attention ka use karke tokens ke beech relationships ko capture karta hai aur training ke time parallel processing ko efficiently support karta hai.

Self-attention - intuition

Har token ke liye attention decide karta hai ki baaki tokens mein se kis information ko kitni importance deni chahiye. Isse model long-range relationships, references aur syntax jaise patterns better samajh pata hai.

- **Query (Q):** Current token kis information ko dhoond raha hai.
- **Key (K):** Har token kya information offer kar raha hai jisse matching ho sake.
- **Value (V):** Attention select hone par actual information jo aggregate hoti hai.

Core equation: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$.

Multi-head attention

Ek hi attention ke bajay multiple attention heads parallel mein different relationships learn karte hain. Unke outputs combine hote hain aur next layers ko pass kiye jaate hain.

4. Training, Fine-Tuning, and Inference

Stage	Kya hota hai	Fresher ko kya samajhna chahiye
Pretraining	Huge dataset se general language/code patterns learn hote hain	Next-token prediction aur representations
Supervised fine-tuning	Curated input-output examples par extra training	Specific behavior/task improve karna
Preference/alignment training	Useful, preferred aur safer behavior ko encourage karna	Instruction following
Inference	Trained model request par answer generate karta hai	Application se API call

Important: RAG inference time par external context deta hai; fine-tuning model ke parameters ko additional training se adapt karta hai. Private/current knowledge ke case mein RAG often simpler starting point hota hai.

5. Prompt Engineering

Prompt woh instruction + context hai jo hum model ko dete hain. Achha prompt ambiguity kam karta hai aur output ko predictable/usable banata hai.

Practical prompt structure

```

Role / task
Context
Constraints
Input
Output format
Examples (agar useful hon)
```

- **Specific raho:** task aur expected behavior clearly define karo.

- **Constraints do:** length, audience, allowed fields, rules.
- **Structured output use karo:** Software ko answer consume karna ho to JSON/schema useful hai.
- **Few-shot prompting:** Difficult pattern ke liye 2-5 examples de sakte ho.
- **Security prompt se solve nahi hoti:** Authorization aur validation backend code mein enforce karo.

6. Structured Outputs and JSON

LLM ka free-form text software ke liye fragile ho sakta hai. Jahan possible ho, schema-constrained structured output use karo.

```
{
  "question": "...",
  "answer": "...",
  "confidence": 0.0,
  "sources": ["..."]
}
```

- Response ko parse karo.
- Types aur required fields validate karo.
- Malformed output ke liye error handling/retry rakho.
- Arbitrary model-generated code ya commands ko blindly execute mat karo.

7. Embeddings

Embedding text ya supported input ka numerical vector representation hota hai. Similar meaning wale texts suitable embedding space mein generally paas hote hain.

Common uses

- Semantic search
- Document retrieval
- Recommendation / similarity
- Clustering aur deduplication

Common similarity measure: $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$.

Embeddings aur tokens same cheez nahi hain. Tokens model ke processing units hain; embeddings numerical vectors hain jo similarity/retrieval ke liye use hote hain.

8. Vector Databases

Vector database embeddings aur metadata ko store karta hai aur similarity search provide karta hai. Dedicated vector DBs ke alawa kuch general databases bhi vector capabilities dete hain.

Concept	Hinglish meaning
Vector	Embedding ka numerical representation
Metadata	Document ID, title, user, timestamp, permissions etc.
Similarity search	Query vector ke closest vectors dhoondna
Top-k	Sabse relevant k results return karna
Index	Search ko fast banane wali data structure/strategy

Production system mein authorization metadata ka use zaroor karo. Sirf semantic similarity se decide nahi hona chahiye ki user document dekh sakta hai ya nahi.

9. RAG - Retrieval-Augmented Generation

RAG mein model ko sirf apni learned knowledge par depend nahi karna padta. Application pehle relevant external context retrieve karti hai, phir us context ko prompt ke saath LLM ko deti hai.

RAG pipeline

```
Documents
↓
Load / clean
↓
Chunk
↓
Embedding model
↓
Vector store
↓
User query → query embedding
↓
Retrieve top-k chunks
↓
Optional reranking / filtering
↓
Prompt = question + retrieved context
↓
LLM
↓
Answer + citations / source IDs
```

Chunking

Documents ko chhote retrievable chunks mein split karte hain, taaki relevant information accurately mil sake. Chunk size aur overlap use-case par depend karte hain. Headings aur paragraph structure preserve karna helpful hota hai.

Retrieval quality

- **Recall:** Kya relevant information retrieve hui?
- **Precision:** Retrieved results mein kitna content actually relevant hai?
- **Reranking:** Retrieved candidates ko relevance ke basis par dobara reorder karna.
- **Metadata filtering:** User, department, date, document type, access permissions ke basis par retrieval restrict karna.

RAG vs fine-tuning

Need	Usually kya consider karen
Private/current documents	RAG
Source citations chahiye	RAG
Knowledge frequently change hoti hai	RAG
Consistent style/behavior adapt karna	Fine-tuning useful ho sakta hai
Application data ka knowledge change karna	RAG often preferred

10. RAG Failure Modes

- Wrong chunks retrieve ho rahe hain -> chunking, metadata filters, query rewriting, embeddings ya reranking improve karo.

- Correct chunks hain but answer weak hai -> prompt, context formatting, output constraints aur model choice dekho.
- Relevant chunk hi nahi mila -> system ko "not enough information" bolna chahiye.
- Context bahut zyada hai -> fewer/high-quality chunks ya reranking/context compression use karo.
- Documents stale hain -> ingestion/update pipeline aur versioning rakho.
- Unauthorized data retrieve ho raha hai -> authorization retrieval se pehle enforce karo.

Strong interview answer: **"RAG unsupported generation ko reduce karne mein help karta hai kyunki model ko retrieved application data diya jaata hai, lekin RAG correctness guarantee nahi karta. Retrieval quality, source quality, prompt design aur evaluation sab matter karte hain."**

11. Function Calling / Tool Calling

Tool calling mein model aapke application ko request de sakta hai ki koi predefined function execute kare - jaise database search, inventory check, ya ticket create karna. Application arguments validate karke tool execute karti hai.

```

User → LLM
↓
tool request
↓
Application validates arguments
↓
Tool executes
↓
Tool result
↓
LLM
↓
Answer

```

- Tool ko narrow aur well-defined schema ke saath expose karo.
- Arguments ko server-side validate karo.
- Authentication aur authorization enforce karo.
- Sensitive operations ke liye allowlists use karo.
- Tool calls, errors aur latency log karo.

Key distinction: Tool calling LLM aur application function ke beech interface hai. Agent ek broader workflow hai jahan model multiple steps mein decide, act, observe aur repeat kar sakta hai.

12. AI Agents

Agentic system mein LLM goal achieve karne ke liye next action decide karta hai aur tools use kar sakta hai. Simple loop: observe -> decide -> act -> result dekho -> continue ya stop.

When agents make sense

- Task naturally multiple steps ya tools demand karta hai.
- Next action previous result par depend karta hai.
- Success condition aur action space clearly bounded hai.

Agent risks

- Infinite/long loops
- Unexpected tool calls
- High token/API cost

- Prompt injection through tools ya retrieved documents
- Irreversible actions

Fresher project mein deterministic workflow se start karna better hota hai. Jahan genuinely benefit ho, wahi agentic behavior add karo.

13. Prompt Injection and Security

Prompt injection tab hota hai jab untrusted content LLM ko manipulate karne ki koshish karta hai - for example intended instructions ignore karwana ya unintended action karwana. RAG aur tool-using systems mein yeh especially important hai.

- Retrieved documents, webpages, emails aur user input ko **untrusted data** samjho.
- System instructions ko untrusted context se logically separate rakho.
- Authorization ka decision model par mat chhodo.
- Tool arguments validate karo aur permissions backend code mein enforce karo.
- Least privilege follow karo.
- High-impact/irreversible actions ke liye confirmation lo.
- Secrets ko prompts ya client-side code mein mat rakho.

Golden rule: **LLM output ko untrusted input treat karo**. External input ki tarah validation aur defensive engineering apply karo.

14. API Integration Basics

Typical GenAI app mein frontend, backend, model provider, retrieval layer aur database/storage components hote hain.

```
Frontend
↓ HTTPS
Backend API
■■■ Auth / rate limit
■■■ Business logic
■■■ RAG / tools
■■■ Model API
↓
LLM provider
```

- API keys environment variables ya secret manager mein rakho.
- Provider secrets browser/client code mein expose mat karo.
- Timeouts aur retries carefully set karo.
- Provider errors aur rate limits handle karo.
- Logs mein request IDs/latency useful hain, lekin sensitive prompts leak mat karo.
- Streaming use karo jab better time-to-first-token UX important ho.

15. Tokens, Cost, Latency, and Context

Metric	Kya samajhna hai
Input tokens	Model ko bheje gaye tokens
Output tokens	Model dwara generate kiye gaye tokens
Latency	Response aane ka time

Metric	Kya samajhna hai
Throughput	System kitni requests/tokens handle karta hai
Context length	Request mein usable context ki maximum capacity
Cost	Often token usage + provider/model pricing par depend karta hai

- Prompt ko unnecessary long mat rakho aur sirf relevant context retrieve karo.
- Reusable results ke liye caching consider karo.
- Simple tasks ke liye cheaper model, complex tasks ke liye stronger model use kar sakte ho.
- Output limits set karo.
- Guess nahi, metrics measure karo.

16. Memory vs Context

Context woh information hai jo current request mein model ko bheji ja rahi hai. **Application memory** woh data hai jo aap system mein store karte ho aur future turns mein selectively retrieve karte ho. LLM ko automatically har past conversation yaad hai, aisa assume nahi karna chahiye.

Chat app mein common pattern hai: recent conversation + summarized history + relevant long-term user data + current retrieved documents.

17. Evaluation - The Missing Skill

Sirf manually kuch prompts chala ke decide nahi karna chahiye ki GenAI app reliable hai. Representative evaluation set banao.

Useful evaluation dimensions

- Correctness / factuality
- Relevance
- Groundedness in sources
- Retrieval recall / precision
- Format validity
- Safety / refusal behavior
- Latency and cost

Simple evaluation loop

1. 30-100 representative test cases banao.
2. Expected behavior/reference answers define karo.
3. Application run karo.
4. Deterministic checks aur/or evaluators se score karo.
5. Failures inspect karo.
6. Ek component change karo.
7. Evaluation set dobara run karo.

RAG mein retrieval aur generation ko separately evaluate karo. Agar correct chunk retrieve hi nahi hua, sirf prompt change karne se root problem solve nahi hogi.

18. Production Checklist

Area	Checklist
Security	Auth, authorization, secrets, validation, tool allowlists
Reliability	Timeouts, retries, fallbacks, graceful errors
Observability	Logs, traces, latency, token usage, error rates
Cost	Token budgets, caching, model routing
RAG	Chunking, metadata, access control, retrieval evaluation
Quality	Evaluation set, regression tests, human review
UX	Streaming, citations, loading/error states

19. Simple Fresher Project: RAG Chatbot

Ek small document collection - jaise placement handbook, company interview notes, project documentation, ya allowed PDFs - par chatbot build karo.

MVP features

- Documents upload/ingest karo.
- Text extract aur clean karo.
- Documents ko chunks mein split karo.
- Embeddings generate karo.
- Chunks + metadata store karo.
- Query ke liye top-k relevant chunks retrieve karo.
- Retrieved context ko LLM ko do.
- Source name/page reference return karo, jahan possible ho.
- "Not enough information" behavior add karo.

Suggested stack

Python + Flask/FastAPI, koi LLM API, embedding model/API, vector store aur simple React/HTML frontend. Architecture itna simple rakho ki interview mein har component ka purpose explain kar sako.

Interview explanation

"Maine RAG implement kiya jisme chatbot answer generate karne se pehle document collection se relevant chunks retrieve karta hai. Embeddings semantic retrieval ke liye use kiye, metadata store kiya, top relevant chunks ko LLM prompt mein add kiya aur source references return kiye. Jab context insufficient ho, system unsupported answer dene ke bajay limitation batata hai."

20. GenAI Concepts You Should Be Able to Explain

Topic	Interview mein one-line explanation
LLM	Large neural language model jo context ke basis par token sequences generate karta hai.
Token	Model-specific text unit jo processing aur generation mein use hota hai.
Embedding	Numerical vector representation jo semantic similarity capture karne mein help karta hai.
Transformer	Attention-based neural architecture jo modern LLMs ka core hai.

Topic	Interview mein one-line explanation
Attention	Mechanism jo decide karta hai ki kaunse tokens ki information kitni important hai.
RAG	External information retrieve karke usko generation ke context mein use karna.
Vector DB	Embeddings ko efficiently store/search karne wala database system.
Fine-tuning	Additional training se model behavior/parameters ko adapt karna.
Tool calling	LLM ka predefined application function ko structured request karna.
Agent	Model-driven system jo multiple actions/tools ko iterate karke goal solve karta hai.
Hallucination	Unsupported ya incorrect generated content.
Prompt injection	Untrusted input ka model instructions/behavior ko manipulate karne ki koshish.
Grounding	Answer ko trusted external information se constrain/supported karna.
Evaluation	Systematically measure karna ki model/application kitna useful aur reliable hai.

21. Common Fresher Interview Questions

Q: What is GenAI?

A: Aise AI systems jo naya text, code, images, audio etc. generate kar sakte hain.

Q: What is an LLM?

A: Large neural language model jo token sequences ko model karta hai aur text/code generate karta hai.

Q: What is a token?

A: Model-specific text unit; zaroori nahi ki ek complete word ho.

Q: What is an embedding?

A: Numerical vector representation jo semantic similarity aur retrieval mein use hoti hai.

Q: Why use RAG?

A: Private/current/external information ko model answer mein ground karne ke liye.

Q: RAG vs fine-tuning?

A: RAG inference time par context deta hai; fine-tuning additional training se model behavior/parameters adapt karta hai.

Q: What is hallucination?

A: Model ka unsupported ya incorrect claim generate karna.

Q: What is temperature?

A: Sampling parameter jo output randomness control karta hai.

Q: What is vector search?

A: Query embedding ke closest embeddings ko similarity metric se find karna.

Q: What is chunking?

A: Documents ko smaller retrievable pieces mein split karna.

Q: What is reranking?

A: Retrieved candidates ko stronger relevance scoring/model se reorder karna.

Q: What is tool calling?

A: LLM ka predefined application function ko structured arguments ke saath request karna.

Q: What is an AI agent?

A: Aisa system jo multiple steps mein decide, tools use, results observe aur continue kar sakta hai.

Q: How do you secure an LLM app?

A: Model output aur external content ko untrusted treat karke auth, validation, least privilege aur safe tool execution enforce karta hoon.

Q: How do you evaluate RAG?

A: Retrieval quality aur answer quality ko separately test karta hoon using representative cases and regression evaluation.

22. 7-Day Fresher Learning Plan

Day	Focus	Deliverable
1	LLM basics, tokens, transformers, prompting	Core terms bina notes explain karo
2	Embeddings, vector search, databases	Semantic search demo banao
3	RAG architecture, chunking, metadata	Small document set index karo
4	RAG chatbot + citations	End-to-end working chatbot
5	Tool calling, agents, security	Ek safe tool add karo
6	Evaluation, caching, rate limits, observability	Evaluation set + metrics
7	Polish + interview prep	README, architecture diagram, demo, Q&A

23. What You Do NOT Need to Master as a Fresher

- Har Transformer equation ko first-principles se derive karna.
- LLM from scratch train karna.
- Khud ka vector database banana.
- Har model provider ki API yaad karna.
- Har GenAI framework memorize karna.
- Sirf buzzword ke liye project ko "agentic" ya "RAG-powered" bolna.

Depth > buzzwords. Resume par RAG, embeddings, Redis, agents ya vector database likha hai to aapko architecture, data flow aur component ka purpose explain karna chahiye.

24. Recommended Learning Order

Programming -> APIs/JSON -> ML basics -> LLM fundamentals -> prompting -> embeddings -> vector search -> RAG -> structured outputs -> tool calling -> agents -> evaluation -> security -> production optimization.

Fresher ke liye yeh order application-first hai. Isse aap jaldi project build kar sakte ho aur interview ke liye enough conceptual depth bhi develop karte ho.

25. Final Checklist

- Main tokens, context windows, temperature aur hallucinations explain kar sakta hoon.
- Main Transformer aur attention ko high level par explain kar sakta hoon.

- Main backend se LLM API call kar sakta hoon.
- Main embeddings aur cosine similarity samajhta hoon.
- Main vector search aur metadata filtering explain kar sakta hoon.
- Main basic RAG pipeline build kar sakta hoon.
- Main chunking, retrieval, reranking aur citations samajhta hoon.
- Main RAG, fine-tuning, tool calling aur agents ko differentiate kar sakta hoon.
- Main prompt injection samajhta hoon aur model output ko untrusted treat karta hoon.
- Main structured LLM output validate kar sakta hoon.
- Main latency, token cost, caching, retries aur rate limits discuss kar sakta hoon.
- Main evaluation dataset bana kar failures measure kar sakta hoon.
- Main resume par listed har GenAI keyword ko implementation story ke saath defend kar sakta hoon.

One-Minute Interview Summary

"Generative AI applications mein generally LLM ke saath application logic, data aur external tools combine hote hain. Mujhe tokens, context, transformers aur attention jaise LLM basics samajh aate hain aur main model ko API ke through integrate kar sakta hoon. Private ya frequently changing knowledge ke liye main RAG use kar sakta hoon jahan documents ko chunk karke embeddings banaye jaate hain, vector retrieval aur metadata filtering hoti hai, aur retrieved context se grounded answer generate hota hai. Actions ke liye tool calling aur basic agentic workflows samajhta hoon. Saath hi hallucination, prompt injection, authorization, evaluation, latency aur token cost ko consider karta hoon, kyunki production GenAI feature sirf text generate karna nahi, reliable aur secure hona bhi hai."

Note: Model/provider APIs, limits aur pricing time ke saath change ho sakte hain. Implementation ke time current provider documentation check karna chahiye.